

Static Code Analysis Rulebook

Doc Ref: ACQO-DOC-004 | v1.0 | 2025

Autonomous Code Quality & Optimization Intelligence System

1. Overview

This rulebook defines the static analysis rules, code smell definitions, complexity thresholds, and anti-pattern detection criteria applied by the ACQO system across all monitored repositories. All rules are enforced automatically during CI pipeline execution and reported to the developer interface for action.

■ **Current Violations:** `AuthManager.java` (user-auth-service) has a cyclomatic complexity of 19.1 and 7 code smells — the highest in the monitored codebase. Immediate refactoring recommended.

2. Code Quality Metrics Baseline

File	Repository	Cyclomatic Complexity	Maintainability Index	Code Smells	Security Vulns	Test Coverage
PaymentProcessor.java	payment-service	18.4	72.5	5	0	81.2%
data_pipeline.py	analytics-engine	15.2	78.3	3	0	85.6%
AuthManager.java	user-auth-service	19.1	68.7	7	1	74.4%
dashboardController.js	reporting-dashboard	12.3	82.1	2	0	88.9%

3. Cyclomatic Complexity Thresholds

Cyclomatic complexity measures the number of linearly independent paths through a function. Higher scores indicate harder-to-test, harder-to-maintain code:

Complexity Score	Risk Level	Action Required	Review Mode
1 – 5	Low	No action needed	Standard PR review
6 – 10	Moderate	Document complex logic with inline comments	Peer review required
11 – 15	High	Refactoring recommended within 30 days	Tech lead review
16 – 20	Critical	Refactoring mandatory before next release	Architecture review
> 20	Severe	Block merge — refactor before re-submission	ACQO agent escalation

4. Code Smell Definitions

Code Smell	Definition	Detection Rule	Severity
Long Method	Function exceeds 40 lines (Java) / 30 lines (Python)	LOC per function > threshold	Medium

Code Smell	Definition	Detection Rule	Severity
Large Class	Class with > 500 LOC or > 20 public methods	Class size analysis	High
Feature Envy	Method uses data from another class more than its own	Cross-class call ratio	Medium
Data Clump	Same group of variables appearing together in 3+ places	Repeated parameter clusters	Low
Primitive Obsession	Using primitives instead of domain objects	Type usage pattern analysis	Low
Switch Statement	Complex switch/if-else chains replacing polymorphism	Branch depth > 4 levels	Medium
Dead Code	Unreachable or unused functions, variables, imports	Reference graph analysis	High
Magic Numbers	Unexplained numeric literals in logic	Literal detection in conditions	Low

5. Maintainability Index

The Maintainability Index (MI) is a composite metric (0–100) that combines cyclomatic complexity, lines of code, and Halstead volume to estimate how maintainable a module is:

MI Score Range	Rating	Meaning	Recommended Action
85 – 100	Excellent	Highly maintainable code	No action required
65 – 84	Good	Acceptable maintainability	Monitor at next review cycle
40 – 64	Fair	Degrading maintainability	Schedule refactoring sprint
0 – 39	Poor	Very hard to maintain	Mandatory refactoring before feature work

6. Security Vulnerability Patterns

- **SQL Injection (CWE-89):** Raw string concatenation in SQL queries. Detected in AuthManager.java (1 active violation). Block merge until fixed.
- **Hardcoded Credentials (CWE-798):** String literals matching password/secret patterns in source files. Auto-fail CI on detection.
- **Insecure Deserialization (CWE-502):** Deserializing untrusted data without type validation. Flag in code review.
- **Path Traversal (CWE-22):** User-controlled file path input without canonicalization. High severity; block deploy.
- **Weak Cryptography (CWE-327):** Usage of MD5, SHA-1, DES, or ECB mode in crypto operations. Replace with SHA-256+.

7. Code Duplication Detection Rules

Duplication is flagged when identical or structurally similar code blocks appear in two or more locations:

Duplication Type	Detection Method	Threshold	Action
Exact clone	Token-level hash comparison	6+ identical lines	Extract to shared utility
Near-miss clone	AST structural similarity	>80% structural match	Review for extraction
Semantic clone	Embedding similarity (vector)	Cosine sim > 0.90	Agent recommendation

Duplication Type	Detection Method	Threshold	Action
Cross-file duplication	File-level scan (code_duplication_flag)	TRUE flag set	Refactoring ticket raised

Static Analysis Pipeline



✓ Recommended Practices	✗ Prohibited Practices
✓ Enforce complexity thresholds as hard CI gates	✗ Suppress static analysis warnings without documented justification
✓ Treat security vulnerability count = 0 as a non-negotiable gate	✗ Allow dead code to accumulate — it becomes confusion and risk
✓ Track maintainability index trends over time, not just snapshots	✗ Deploy a module with active security vulnerability flags